

Chap 3

(1 or 2 Q) (8-15) marks

RTL and M.O.L

~~Q1~~

Q1) What is micro-operation?

Ans: Micro operations are basic tasks the computer can perform.

- They are small, specific actions like adding, subtracting, comparing or moving data.
- Computer instructions are broken down into sequence of micro operations.
- By performing a sequence of micro-operations that move and modify data, a sequential digital system performs a larger task.
- Example: a CPU performs a sequence of micro-operations to fetch, decode and execute an instruction.

Q2) Define RTL.

- RTL stands for Register Transfer Language.
- It is a way of describing how data moves between registers in a computer's digital circuits using simple instructions.
- Engineers use RTL to plan and design digital circuits before physically building them.
- RTL is like making a blueprint of digital circuit.
- It helps engineers to visualize how information is processed and moved around within a computer's hardware.

3) How micro-operations can be expressed in terms of the RTL? Illustrate with example.

Ans: Micro-operations can be expressed in RTL by describing the transfer of data between registers and the logical operations performed on that data.

Example: Micro-operation: Add the contents of register X and Y and store the result in register X.

RTL representation:

$$X \leftarrow X + Y$$

Explanation:

- The arrow ($\leftarrow$) represents the transfer of data from the right side to left side.
- 'X' and 'Y' are registers.
- The '+' sign represents the addition operation being performed on the data from registers 'X' and 'Y'.

The RTL representation captures the micro operation of adding the contents of 'X' and 'Y' and storing the result in 'X'.

clk $\rightarrow$ clock signal
LD $\rightarrow$ load signal

Date: _____

Page: _____

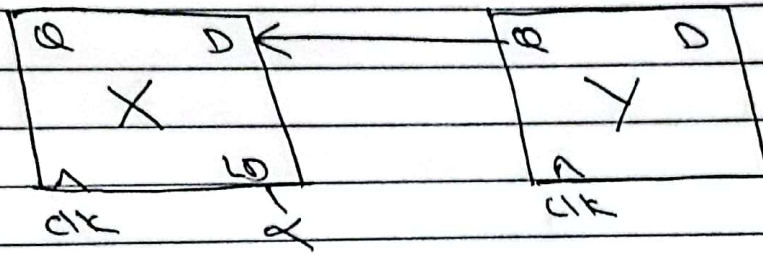
Example 2:

Consider that, the transfer $X \leftarrow Y$ occurs when control signal α is high in RTL.

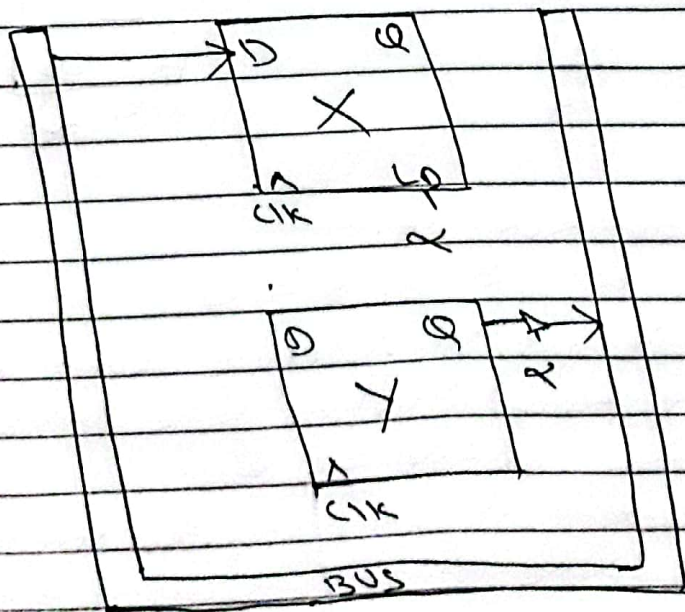
$\alpha: X \leftarrow Y$

RTL implementation

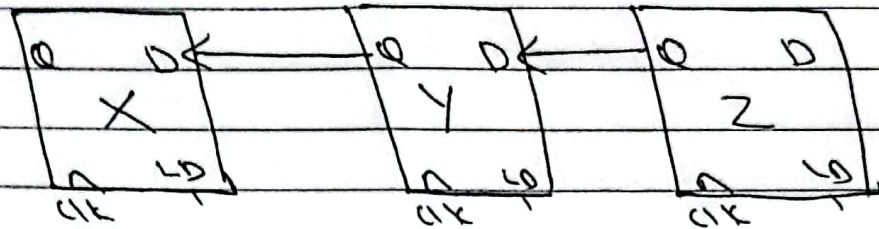
a) Direct connection



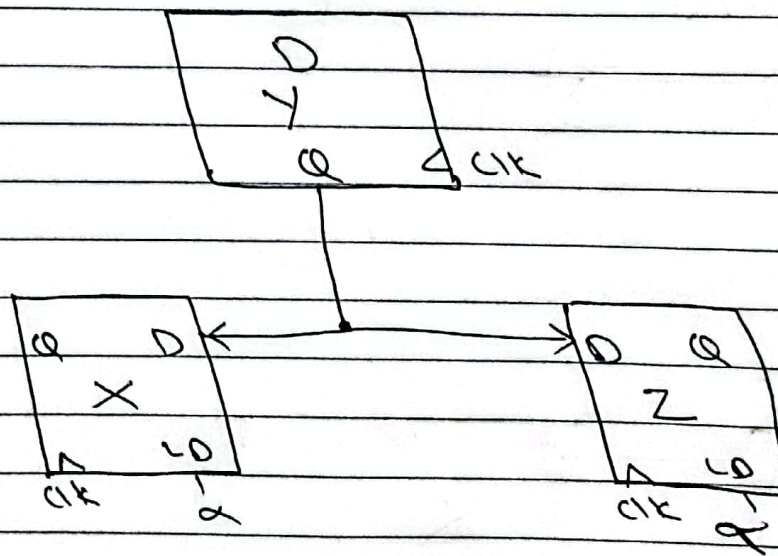
b) Bus connection



Implementation of the data transfer
 $\alpha: X \leftarrow Y, Y \leftarrow Z$



Implementation of the data transfer
 $\alpha: X \leftarrow Y, Z \leftarrow Y$



Note: Implementation of
 $\alpha: X \leftarrow Y, X \leftarrow Z$

→ This is not possible, because same register cannot write two datas/multiple datas at the same time from multiple registers.

→ However, content of same register can be copied into two/multiple other registers at the same time.

114) Write down the arithmetic and logical operations in RTL form. & design the circuitry.
OR

Write RTL codes for arithmetic operations with circuitry.
OR

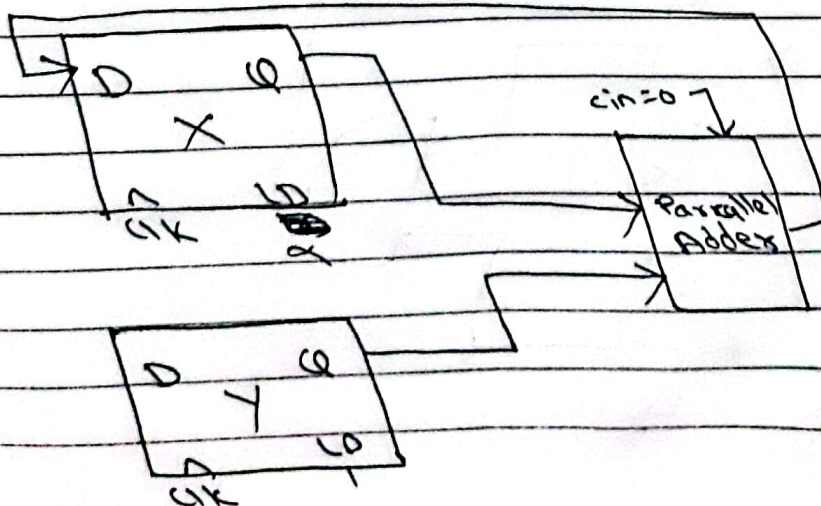
Show the implementation of addition and subtraction using parallel adder & one.

Ans: Arithmetic and logical micro-operations are:

Operation	Example
Add	$X \leftarrow X + Y$
Subtract	$X \leftarrow X - Y$ or $X \leftarrow X + Y' + 1$
Increment	$X \leftarrow X + 1$
Decrement	$X \leftarrow X - 1$
AND	$X \leftarrow X \wedge Y$ or $X \leftarrow XY$
OR	$X \leftarrow X \vee Y$
XOR	$X \leftarrow X \oplus Y$
NOT	$X \leftarrow X'$

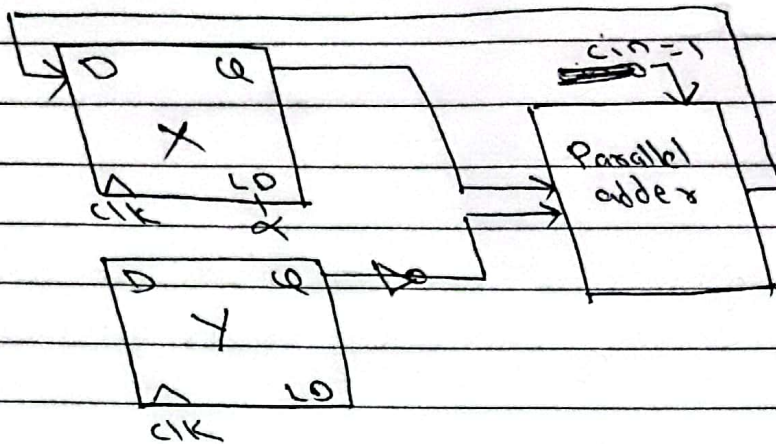
Implementation:

1) Addition ($X: X \leftarrow X + Y$)



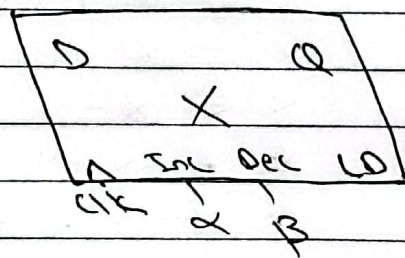
2) Subtraction: ~~2~~

$$\alpha: (X \leftarrow X - Y) \text{ or } (X \leftarrow X + Y' + 1)$$

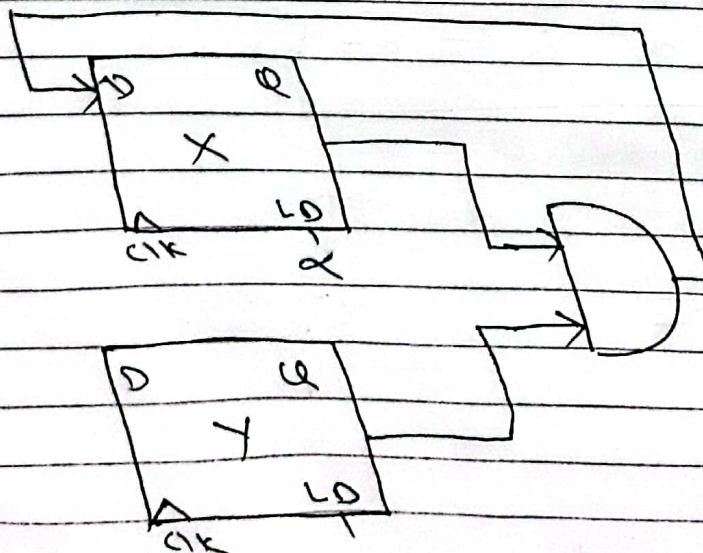


3) Increment & Decrement

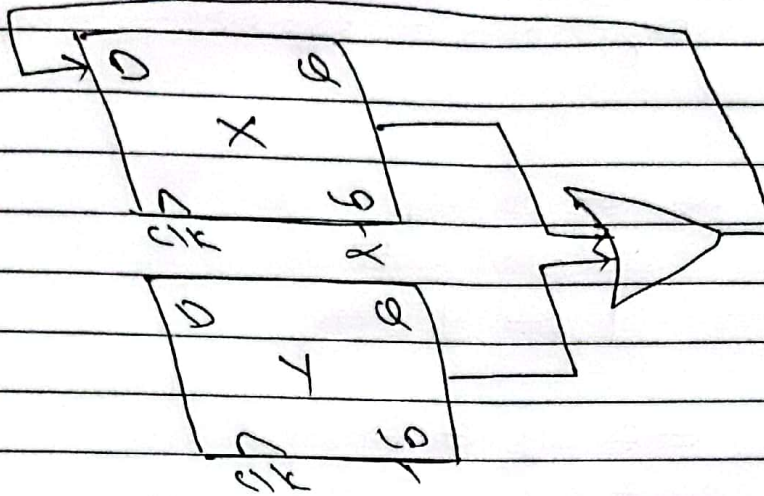
$$\alpha: X \leftarrow X + 1, \beta: X \leftarrow X - 1$$



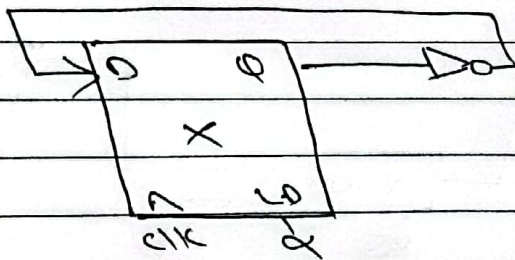
4) AND ($\alpha: X \leftarrow X \wedge Y$)



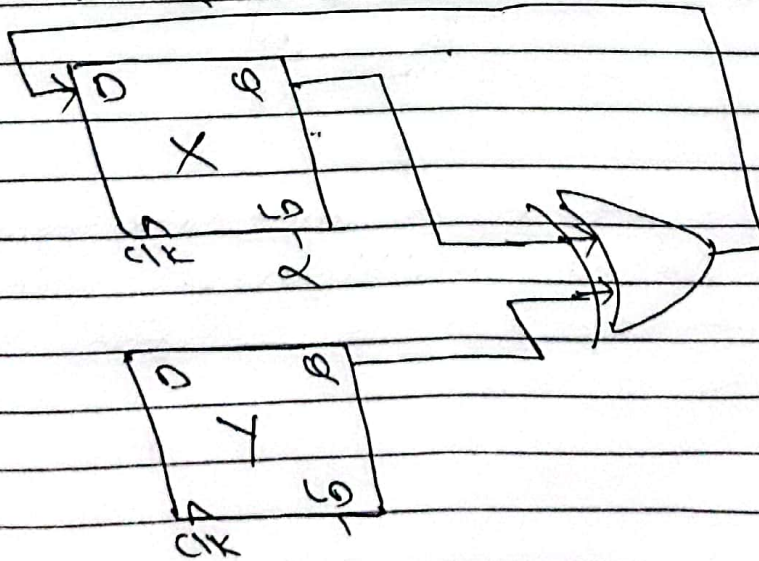
5) OR ($\alpha: X \leftarrow X \vee Y$)



6) NOT ($\alpha: X \rightarrow X'$)



7) XOR ($\alpha: X \leftarrow X \oplus Y$)



5) Design the system for the given RTL codes using direct connection and bus:

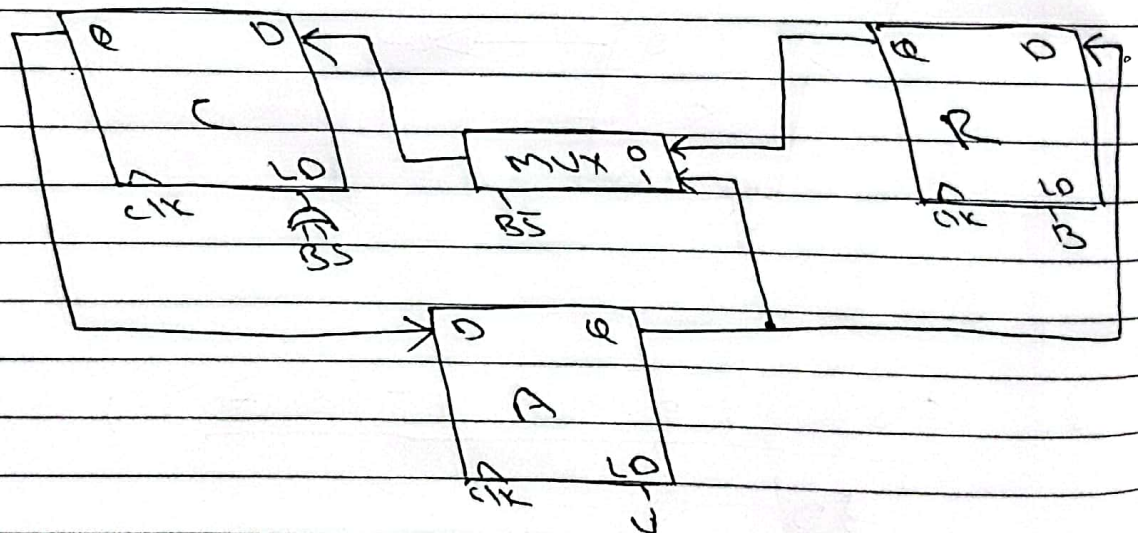
~~Q: $C \leftarrow A$, $R \leftarrow A$~~
~~V: $A \leftarrow C$~~
~~S: $C \leftarrow R$~~

B: $C \leftarrow A$, $R \leftarrow A$

V: $A \leftarrow C$

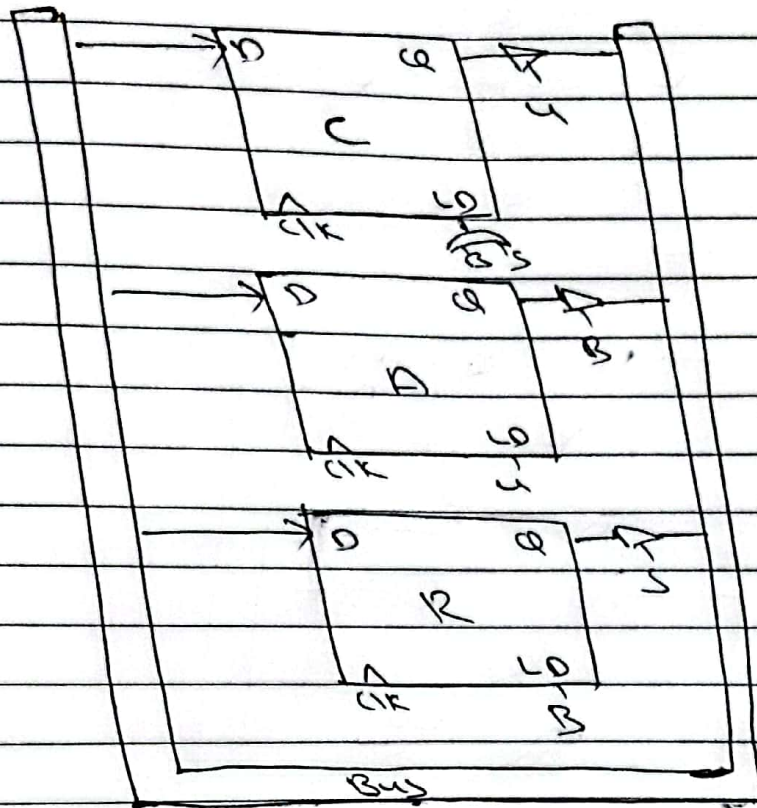
S: $C \leftarrow R$

1) Direct Connection



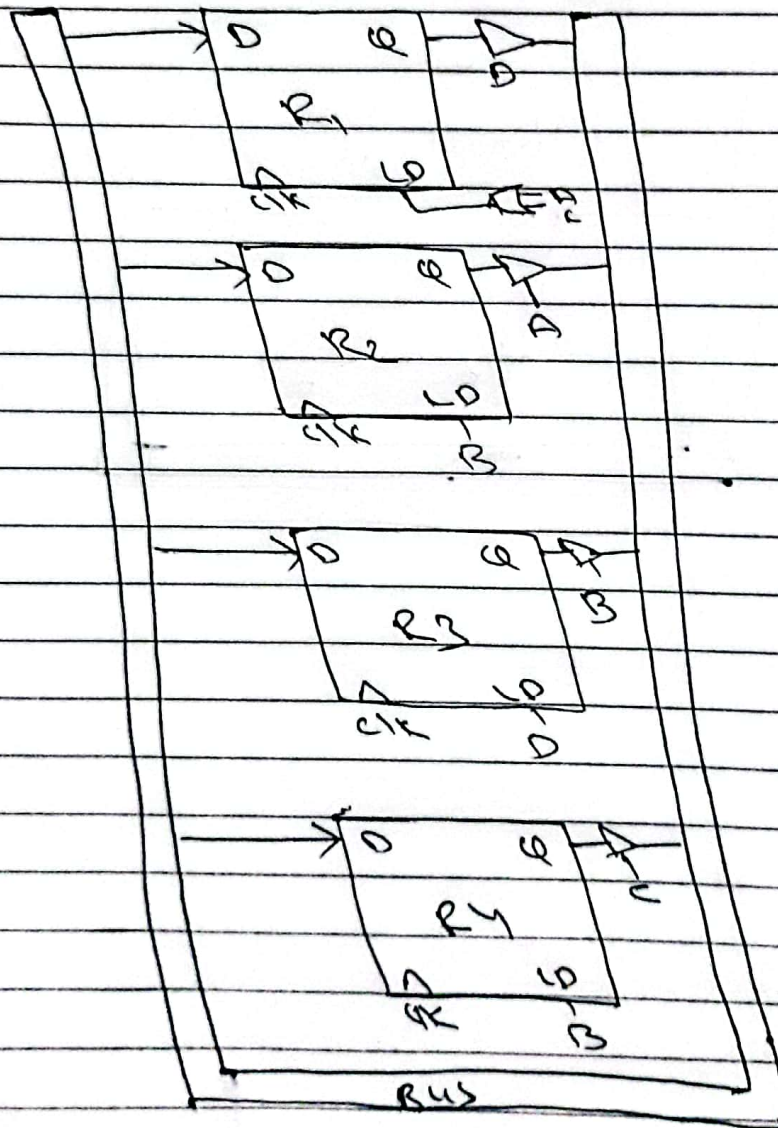
Note: If a register has to receive data from two registers, it uses multiplexer, which takes two data, and based on condition it passes only one data.

2) Bus Connection

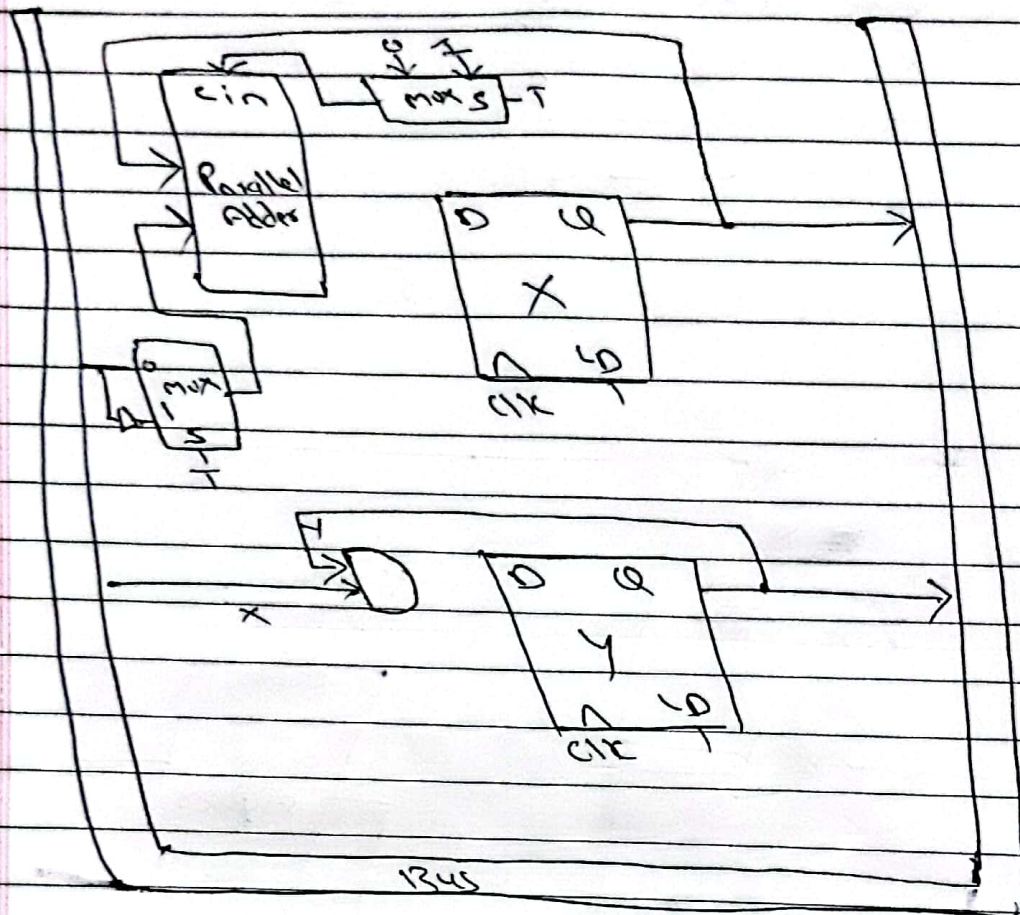


(Design the system)

16) Implement the given RTL code using bus.

A: $R1 \leftarrow R2$ B: $R2 \leftarrow R3, R4 \leftarrow R3$ C: $R1 \leftarrow R4$ D: $R3 \leftarrow R1$ 

ii) Bus Connection



(theory not asked yet, but understand for numerical)

8) Explain all shift micro operations.

→ There are four types of shift microoperations

	operation	notation
i)	Linear shift → a) linear shift left	$(sl)(x)$
	b) linear shift right	$(shr)(x)$
ii)	Circular shift → a) circular shift left	$(cil)(x)$
	b) circular shift right	$(cir)(x)$
iii)	Arithmetic shift → a) arithmetic shift left	$(ashl)(x)$
	b) arithmetic shift right	$(ashr)(x)$
iv)	Decimal shift → a) decimal shift left	$(dshl)(x)$
	b) decimal shift right	$(dshr)(x)$

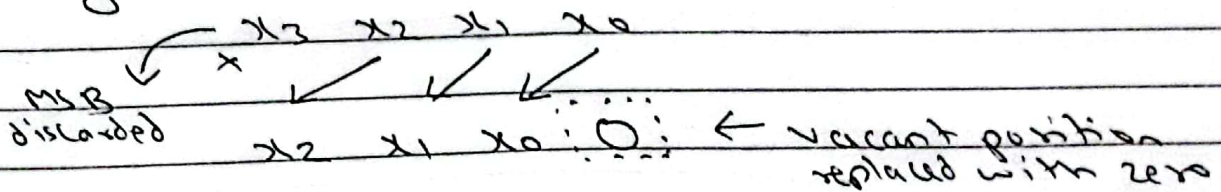
→ They are explained below:-

i) linear shift left: $sl(x)$

→ shift all the bits to left

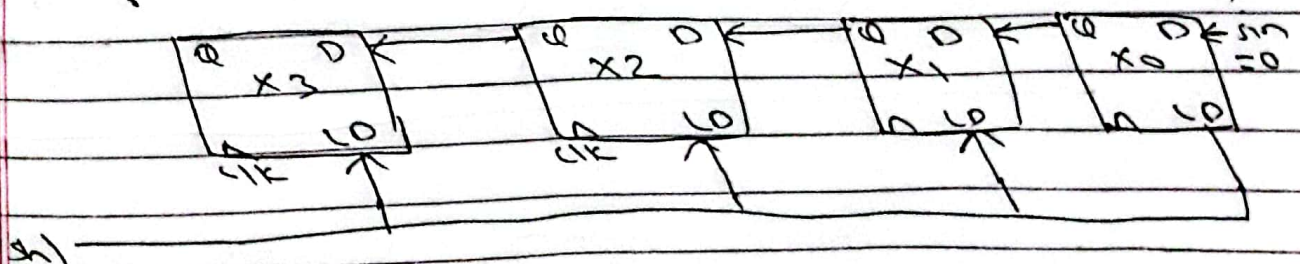
→ MSB is discarded, and we load zero at LSB which is a vacant.

Eg:



Implementation

serial input

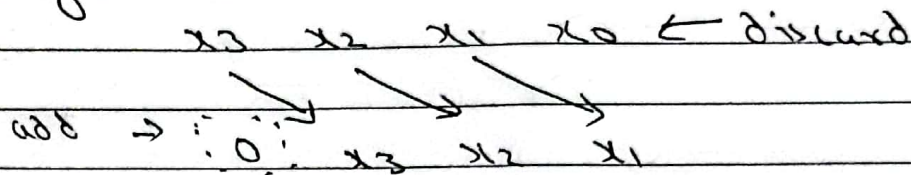


ii) linear shift right $(shr)(x)$

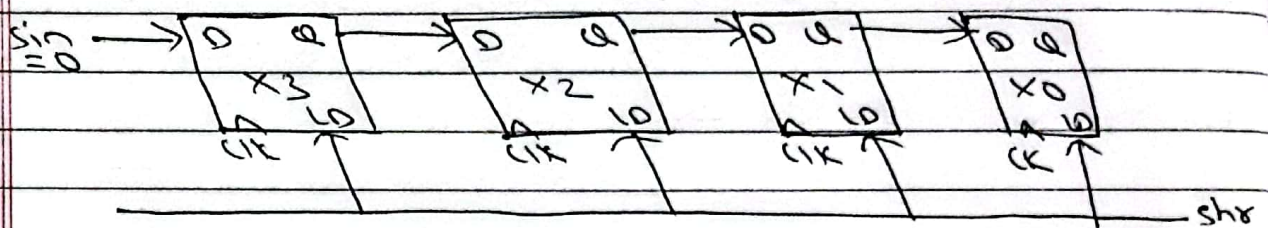
→ shift all the bits to right.

→ LSB is discarded and we load zero at MSB.

Eg:



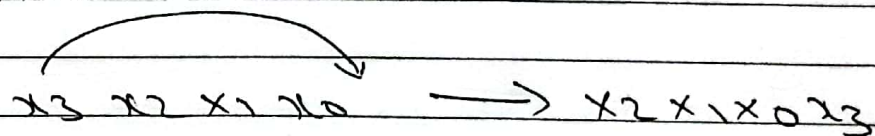
Implementation:



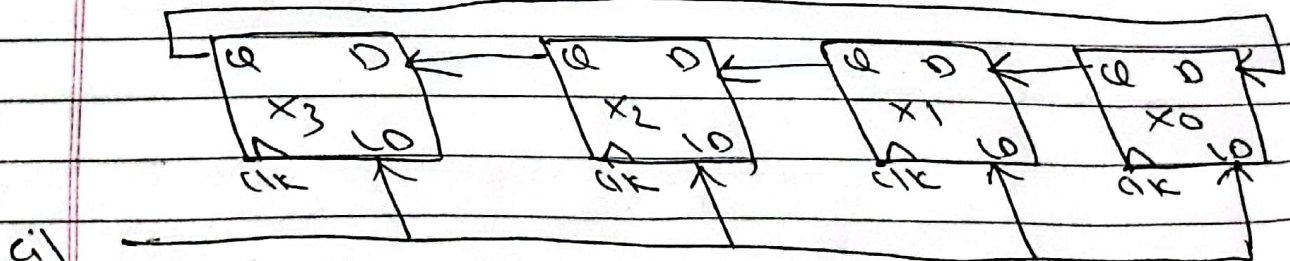
iii) Circular shift left: $cil(x)$

→ all bits are ^{shifted} ~~circulated~~ to left and the bit at the beginning is circulated back at the end position.

Eg:



Implementation:

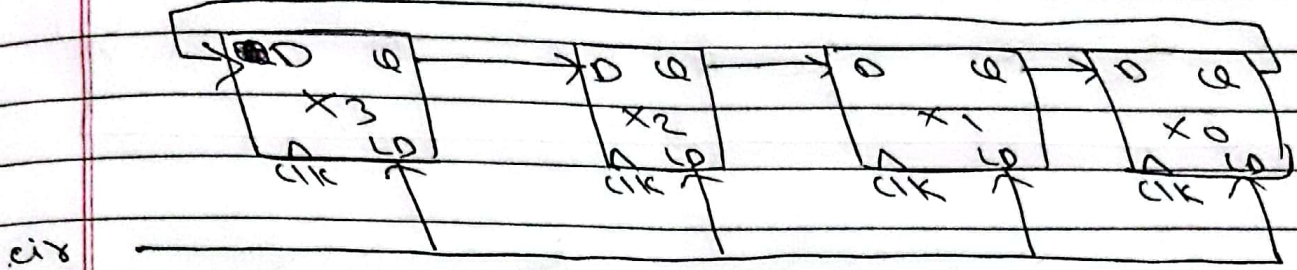


iv) Circular shift right: $cir(x)$

→ all bits are shifted to right and the bit at the last is circulated at the 1st position.

eg: $x_3 \ x_2 \ x_1 \ x_0 \rightarrow x_0 \ x_3 \ x_2 \ x_1$

Implementation:



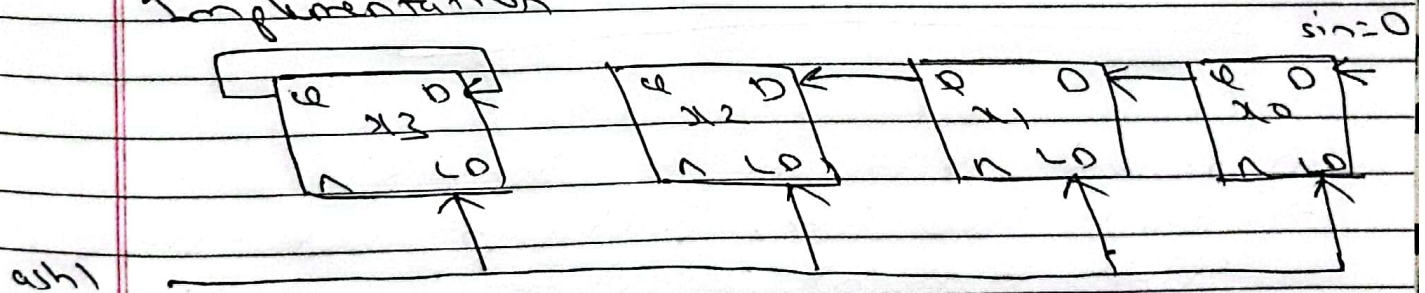
v) arithmetic shift left (ashl)

- All bits are shifted to left except the bit at the beginning, which is the sign bit.
- zero is added at the vacant ~~position~~ place.

eg:

$x_3 \ x_2 \ x_1 \ x_0$
sign bit $\downarrow$ $\swarrow$ $\searrow$
 $x_3 \ x_1 \ x_0 \ '0'$ → zero added

Implementation



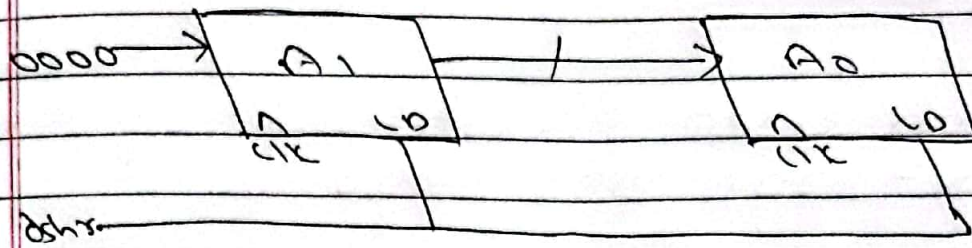
vi) arithmetic shift right

- first bit remains as it is, all bits are shifted to right and last bit is discarded.

eg:

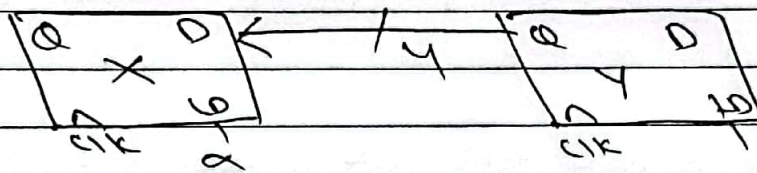
$x_3 \ x_2 \ x_1 \ x_0 \rightarrow$ discarded
sign bit $\downarrow$ $\swarrow$ $\searrow$
 $x_3 \ x_2 \ x_1$

Implementation

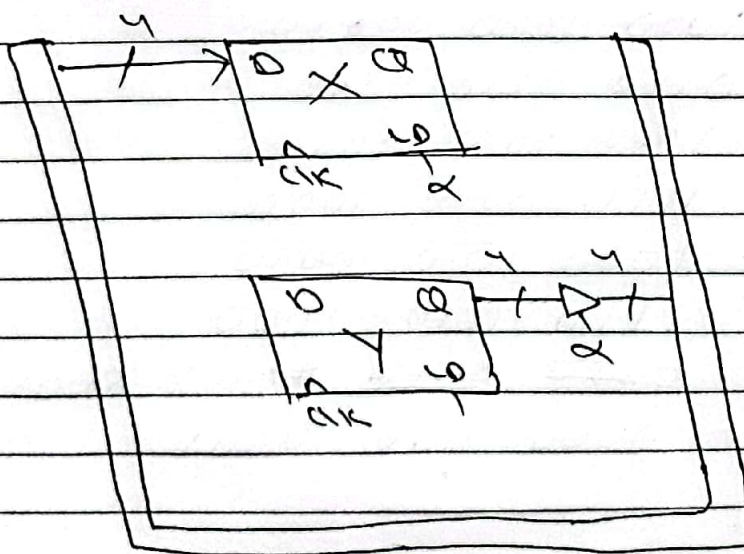


Implementation of a 4 bit data transfer
 $x \leftarrow y$ using (a) direct connection (b) bus

a) direct connection



b) 4-bit bus



9) For $x = 1001\ 1001\ 0000\ 0010$, show the result

9) Perform eight different shift operations on $x = 1001\ 1001\ 0000\ 0010$

shl = 0011 0010 0000 0100

shr = 0100 1100 1000 0001

cil = 0011 0010 0000 0101

cir = 0100 1100 1000 0001

ashl = 1011 0010 0000 00100

ashr = 1100 1100 1000 0001

dshl = 1001 0000 0010 0000

dshr = 0000 1001 1001 0000

10) Perform eight different shift operations on $x = 1100\ 0101\ 0001$

shl = 1000 1010 00010

shr = 0110 0010 1000

cil = 1000 1010 0011

cir = 1110 0010 1000

ashl = 1000 1010 0010

ashr = 1110 0010 1000

dshl = ~~0000~~ ~~0010~~ ~~0010~~ 0001 0000

dshr = 0000 1100 0101

11) Perform eight shift operation on
 $X = 10100100$

$$shl = 01001000$$

$$shr = 01010010$$

$$cil = 01001001$$

$$cir = 01010010$$

$$ashl = 11001000$$

$$ashr = 11010010$$

$$dshl = 01000000$$

$$dshr = 00001010$$

a) show the circuitry for arithmetic right shift and circular left shift operation.

a) write the RTL for shift operations.

a) Design a 4 bit circular right shifting circuit.

Q12. Define HDL

Ans: HDL stands for Hardware Description Language.

- It is a specialized computer language used to design and describe digital circuits and systems.
- HDL allows engineers to specify the behavior and structure of hardware, from basic gates to complex system. ↘ at various levels.
- HDLs are used to create models of digital circuits for simulation and verification.

Q13. Explain VHDL

Ans: VHDL stands for VHSIC (Very High Speed Integrated Circuit) Hardware Description Language.

- It is widely used for designing and simulating digital systems.
- It allows engineers to describe the structure and behavior of circuits.
- Simply, it describes how digital circuit work.
- We can simulate VHDL designs to check if they're correct before making real hardware.
- It is used in creating all sorts of electronics from computer chips to space exploration.
- It is a powerful tool for designing and testing circuits.

Advantages of VHDL

- i) Portability → VHDL designs can be used across different platforms.
- ii) Device independence → VHDL designs are not tied to a specific device, allowing the same design to be implemented on different types of hardware like custom chips, ~~ASICs~~ FPGAs, etc.
- iii) Simulation → VHDL designs can be tested and simulated in software before building the actual hardware.
- iv) System Specification → VHDL let designers work at various levels of detail, from high level system concepts to low-level digital logic, providing flexibility in design and implementation.
- v) Structured Design → VHDL encourages structured and organized design practices, leading to more manageable and modular code.
- vi) Collaboration → It enables collaboration among design teams.

Disadvantages of VHDL

14. VHDL syntax. (Not asked yet)

→ VHDL design codes has 3 primary sections.

i) Library declarations

→ It consists of statements that specify libraries to be accessed.

→ The most commonly used library is IEEE.

```
library IEEE;  
use IEEE.std_logic-1164.all;
```

ii) Entity section

→ It defines the names, input-output signals and modes of a hardware module.

Syntax:

```
entity entity-name is  
  
    port declaration;  
end entity-name;
```

iii) Architecture Section

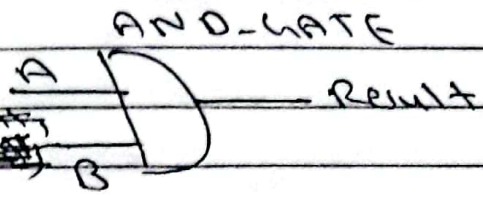
→ It contains architecture

Syntax:

```
architecture architecture-name of entity-name;  
    architecture declaration-part;  
  
begin  
    statements;  
end architecture-name;
```


15. Write a VHDL code for AND Gate.
(Not asked yet)

library IEEE;
~~use IEEE.std_logic_1164.all;~~
library IEEE;
use IEEE.std_logic_1164.all;



entity AND-GATE is

port (A, B : in std_logic;
Result : out std_logic);

end AND-GATE;

architecture behavioral_and of AND-GATE is
begin

Result <= A and B;

end behavioral_and;

116. Write a VHDL code to generate a function
 $F = AB + A'B'C$.

→

```
library IEEE;
```

```
use IEEE.std_logic_1164.all;
```

```
entity combinational-circuit is
```

```
port (
```

```
    A, B, C: in std_logic;
```

```
    F: out std_logic);
```

```
end combinational-circuit;
```

```
architecture dataflow of combinational-circuit is  
begin
```

```
    F <= (A and B and C) or
```

```
        (not A) and (not B) and (not C));
```

```
end dataflow;
```


17. Write a VHDL code for generating the combinational circuit with three inputs A, B and C and an output F
- $$F = (A \wedge B) \vee (A' \wedge B')$$

→

```
library IEEE;
use IEEE.std_logic-1164.all;

entity combinational_circuit is
    port (
        A, B, C: in std_logic;
        F: out std_logic);
end combinational_circuit;

architecture dataflow of combinational_circuit is
    begin
        Y <= (A and B) or ((not A) and (not B));
    end dataflow;
```

18. Write the VHDL code for generating the circuit for the function

i) $F(x, y, z) = \sum (1, 3, 4, 6)$

ii) $F(A, B, C) = \sum (1, 3, 6, 7)$

iii) $F(A, B, C) = \pi (0, 5, 7)$

} One que each

} 7 marks

→ solve separately.

Here, $\sum \rightarrow$ represents min terms and
 $\pi \rightarrow$ represents max terms

Input Variables			Min terms	Max terms
A	B	C	$\sum m$	$\pi \sum M$
0	0	0	$A'B'C' (m_0)$	$A+B+C (M_0)$
0	0	1	$A'B'C (m_1)$	$A+B+C' (M_1)$
0	1	0	$A'BC' (m_2)$	$A+B'+C (M_2)$
0	1	1	$A'BC (m_3)$	$A+B'+C' (M_3)$
1	0	0	$AB'C' (m_4)$	$A'+B+C (M_4)$
1	0	1	$AB'C (m_5)$	$A'+B+C' (M_5)$
1	1	0	$ABC' (m_6)$	$A'+B'+C (M_6)$
1	1	1	$ABC (m_7)$	$A'+B'+C' (M_7)$

$$1.) F(x, y, z) = \Sigma(1, 3, 4, 6)$$

$$\{F(A, B, C) = \Sigma(1, 3, 4, 6)\}$$

$$\rightarrow F = x'y'z + x'yz + xy'z' + xyz'$$

VHDL code

```
library IEEE;
use IEEE.std_logic-1164.all;
```

```
entity combinational-circuit is
```

```
port (
```

```
    in x, y, z : in std_logic;
    f : out std_logic);
```

```
end combinational-circuit;
```

```
architecture dataflow of combinational-circuit is
```

```
begin
```

```
    f <= ((not A) and (not B) and C) or
          ((not A) and
```

```
          ((not x) and (not y) and z) or
          ((not x) and y and z) or
          (x and (not y) and (not z)) or
          (x and y and (not z));
```

```
end dataflow;
```

ii) $F(A, B, C) = \Sigma(1, 3, 6, 7)$
 → same like (i)

iii) $F(A, B, C) = \Pi(0, 5, 7)$
 →

$$F(A, B, C) = (A + B + C)(A' + B + C')(A' + B' + C')$$

library IEEE;

use IEEE.std_logic-1164.all;

entity combinational-circuit is

port (

A, B, C : in std_logic;

F : out std_logic);

end combinational-circuit;

architecture dataflow of combinational-circuit is
 begin

F <= (A or B or C) and

(not A) or B or (not C) and

(not A) or (not B) or (not C);

end combinational-circuit;